ActionDesk

Product Requirements Document + Technical Architecture | Version 1.0 | 13 October 2026

From requests to results. • AI-assisted internal operations platform

Status: proposed design, not implemented. This document defines initial acceptance criteria and consciously defers later capabilities.

Executive decision

Build a usable, single-organization internal operations application that turns free-text requests into tracked service tickets, expense approvals, and employee leave requests. The AI interprets and summarizes; application code owns identity, permissions, business rules, transitions, and side effects. First deploy as a low-cost pilot on AWS EC2 with Docker Compose, PostgreSQL, Redis, Groq primary and Gemini fallback. Keep manual workflows fully usable if both AI providers fail.

1. Product goals and boundaries

Measure	MVP target / decision
Primary users	Requester, Assignee, Approver, Organization Admin
Organizations	One seeded organization in first deployment; schema tenant-aware from day one
Core workflows	Service issue, expense purchase/payment, leave request
Input channels	Web form and signed inbound webhook (pilot); email ingestion deferred
AI functions	Type/field extraction, safe clarification, summaries, daily briefing
Out of scope v1	Visual workflow designer, actual payments, payroll, live WhatsApp, mobile app, arbitrary SQL/agent actions
North-star outcome	Requests no longer disappear into chat; owners, due dates, approval status, and history are visible

2. Users, roles and authorization

Role	Can do	Cannot do
Requester	Create and view own requests, comment, supply missing fields, withdraw eligible requests	Approve own expense/leave, view other employees' confidential requests
Assignee	View tasks explicitly assigned to them/team, update service progress, comment	Approve expenses without approver role, change routing policy
Approver	Review requests assigned to approval queue; approve/reject with reason	Approve their own request, alter audit history

Admin	Manage organization users/teams/assignment policies, inspect organization audit/reporting	Bypass audit or modify historical event records

Permissions are checked against trusted authenticated identity on every API endpoint AND before returning any object. Every query scopes organization_id; employee leave and financial details require finer visibility rules. Avoid treating an LLM proposal as an authorization decision.

3. MVP user journeys

3.1 Submit any request

1. User signs in, selects a workflow or enters a free-text request.
2. API validates input, creates a persisted draft/submitted record, returns a stable request ID.
3. Background AI worker classifies/extracts candidate fields; Pydantic validates structure.
4. If required information is missing, status becomes NEEDS_INFO with clear questions; user can provide data manually.
5. Deterministic rules derive queue, approvals, due date, and next transition.
6. Activity timeline records state changes, actor, reason and timestamps.

3.2 Service request

Employee reports an operational issue. Required: title or description; optional location, urgency, preferred due time and attachments (deferred initially). Admin-defined rules determine department/assignee and SLA. Assignee marks in progress and resolved; requester confirms closure or admin closes with reason.

3.3 Expense request

Requester submits vendor, amount, currency, purpose and optional due date. Missing vendor/amount must be clarified, never guessed. Policy example: amounts > PKR 50,000 require manager approval; > PKR 200,000 require manager then finance approval. These are seed-demo rules and not universal accounting policies. No money is moved.

3.4 Leave request

Requester gives start/end dates and leave type, with optional context. Validate date order and overlaps (flag, do not automatically deny). Manager approves/rejects, reason recorded. No payroll/calendar integration in v1.

3.5 Manager daily briefing

Backend gathers authorized overdue items, impending SLA breaches and pending approvals through deterministic SQL queries. The LLM may summarize only the supplied structured facts and cite internal request IDs. On LLM failure return the same facts as a plain deterministic list.

4. Workflow definitions and transition controls

Type	States (happy path)	Alternative states
Service	SUBMITTED → TRIAGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED	NEEDS_INFO, REOPENED, CANCELLED
Expense	DRAFT → SUBMITTED → MANAGER_REVIEW →	NEEDS_INFO, REJECTED, WITHDRAWN

	[FINANCE_REVIEW] → APPROVED → COMPLETED	
Leave	DRAFT → SUBMITTED → MANAGER_REVIEW → APPROVED	NEEDS_INFO, REJECTED, WITHDRAWN

- Only transition_service.transition(actor, request_id, action, expected_version) changes workflow states.
- Each transition checks current state, requester/approver separation, required fields, organization visibility and role.
- Use transaction + row lock or optimistic version column to avoid double approvals and lost updates.
- Persist request state and immutable workflow event in the same transaction. Use a transactional outbox for external notifications/jobs.
- Retryable jobs must be idempotent; each webhook has unique (organization_id, source, external_event_id).
- Do not auto-complete/approve any expense or leave based solely on model output.

5. AI requirements and output contracts

Groq is primary and Gemini fallback. Use a provider-independent interface with structured extraction per request_type and a deterministic parsing/validation layer. A provider 200 response is not success until the schema and domain validators pass.

AI capability	Input	Validated output / fail-safe
Classification	Free-text, allowed request categories	type enum, urgency suggestion, confidence 0–1, evidence spans; uncertain → manual selection
Extraction	Text + selected type + locale/timezone	typed fields, missing_fields, ambiguities, each value grounded in quoted user text
Clarification	Required schema + known fields	questions limited to truly missing/conflicting values; do not guess
Summary	Authorized request comments/events	brief summary with request IDs, not new factual claims
Daily briefing	SQL-derived request counts/items	bounded concise prose; deterministic list on failure

Example contract (expense extraction): request_type="expense"; vendor: string|null; amount: Decimal|null; currency: enum; purpose: string|null; due_date: ISO date|null; missing_fields: list[enum]; warnings: list[string]; evidence: mapping of extracted field to substring. Monetary values use Decimal in business code, never float.

Provider failure policy: short timeout; limited retry only for transient errors; circuit-break or temporarily stop routing to an exhausted provider; one bounded fallback attempt; use manual review after both fail. Enforce model-call budget per request and global quota, log token/cost estimates but not raw personal content. Free-tier availability and limits may change; verify at integration time.

6. Functional screens

Screen	MVP contents
Sign in	Secure sign-in; demo accounts may be seeded locally

	but not distributed with shared public credentials
Inbox	Filters by type, status, priority, owner, date; pagination; scoped visibility
New request	Type selector + natural language; generated fields editable before submission
Request detail	Validated fields, state, assignee, comments, history, AI provenance, permissible actions
My approvals	Pending items, reasons, explicit approve/reject confirmation
Dashboard	Open, overdue, pending approvals, SLA risks, resolution times from SQL
Operations briefing	Structured attention list + optional generated summary
Administration	Seeded teams, member roles, assignment/SLA/rule configuration (forms, no visual designer)

7. Technical architecture

Browser (Next.js) → FastAPI REST → PostgreSQL (source of truth). After transaction commit, outbox relay queues jobs in Redis; worker invokes AI providers or notification adapter; result is validated and persisted back to PostgreSQL. Scheduled worker checks SLA deadlines. Webhook ingress verifies HMAC and deduplicates before outbox commit. Dashboard metrics are aggregated by SQL. Reverse proxy terminates HTTPS.

One-EC2 pilot Docker Compose services: reverse-proxy, web, api, worker, scheduler (or worker scheduled task), postgres, redis. This is a single-host failure domain by design; nightly off-host backups and tested restore are minimum for real pilots. Avoid using EC2 public HTTP without TLS.

8. Suggested repository structure

- apps/web/ — Next.js TypeScript pages/components/API client
- apps/api/app/api/ — FastAPI routers, auth dependencies, request/approval/webhook endpoints
- apps/api/app/domain/ — workflow transition rules, permissions, policies, typed domain models
- apps/api/app/services/ — request, approval, audit, analytics, AI orchestration
- apps/api/app/integrations/ — Groq/Gemini adapters, notification/webhook clients
- apps/api/app/db/ — SQLAlchemy models, Alembic migrations, repository methods
- apps/worker/ — AI extraction, outbox relay, email and SLA jobs
- packages/contracts/ — OpenAPI-generated TypeScript types or shared schemas
- data/seed/ — deterministic seed specifications and synthetic corpora
- data/evaluation/ — held-out test inputs with verified gold labels
- tests/ — unit, API, integration, e2e, authorization, adversarial
- infra/ — compose, reverse proxy, EC2 bootstrap, deploy and backup scripts
- docs/ — architecture decisions, runbook, security, demo workflow

9. Data model (logical schema)

Table	Important fields / constraints
organizations	id, slug UNIQUE, name, created_at

users	id, organization_id, email unique per org, password_hash / oidc_subject, status
memberships	organization_id, user_id, role; role enum and unique pair
teams	id, organization_id, name; team_members unique pair
requests	id, organization_id, type, title, raw_text, status, priority, created_by, assigned_to, team_id, due_at, version, timestamps
request_fields	request_id UNIQUE, validated JSONB per type, schema_version
request_events	id, request_id, sequence UNIQUE, actor_id nullable, action, from_state, to_state, reason, at
approvals	id, request_id, step, approver_id/role, status, decision_reason, decided_at; unique active step
comments	id, request_id, author_id, body, created_at
notifications	id, recipient_id, request_id, channel, status, sent_at
webhook_events	id, org, source, external_event_id, payload_hash, processed_at; UNIQUE(org,source,external_event_id)
outbox_events	id, event_type, payload, created_at, dispatched_at, attempt_count
ai_runs	id, request_id, task, provider, model, prompt_version, schema_version, result_status, latency_ms, tokens, error_code
audit_logs	id, org, actor_id, action, object_type/id, metadata, at; append-only by app permission

Indexes: requests(org, status, created_at); requests(org, assigned_to, status); requests(org, due_at); approvals(approver_id, status); request_events(request_id, sequence); outbox(dispatched_at, created_at). All timestamps UTC. Avoid storing full LLM prompts or sensitive free-text in telemetry.

10. API contract (initial)

Method/path	Purpose / constraints
POST /auth/login; GET /auth/me	Session or JWT, secure cookie preferred for same-origin web
GET /requests	Scoped paginated filters; no cross-org records
POST /requests	Validate and create request; Idempotency-Key for external/client retries
GET /requests/{id}	Request + allowed transitions, structured fields, timeline
PATCH /requests/{id}/fields	Edit fields subject to role/state/version
POST /requests/{id}/transitions	Expected version + explicit action; reject invalid transitions
POST /requests/{id}/comments	Comment as authorized participant
GET /approvals; POST /approvals/{id}/decision	Explicit approve/reject and reason; block self-approval

GET /dashboard/summary	Authorized SQL metrics and overdue list
POST /briefings/today	Authorized snapshot + AI summary/manual fallback
POST /webhooks/ingest/{source}	Raw-body HMAC, timestamp/replay protection, dedupe, 202 ACK
GET /notifications; POST /notifications/{id}/read	In-app notification state
GET /health; GET /ready	Liveness vs dependency readiness; no secrets

Use RFC 7807-style error bodies, X-Request-ID, OpenAPI spec and explicit 401/403/404/409/422/429 behavior. Do not implement unnecessary generic CRUD endpoints that circumvent domain transitions.

11. Data and evaluation design

Seed ~1 organization, 75 fictional users, 7 departments, 2,800 historical requests (about 1,500 service, 800 expense, 500 leave), realistic events/comments and 300 held-out extraction cases. These are proposed seed targets, not actual measured production figures. A smaller 100-request seed should support local development first.

- Generate canonical structured facts first, then paraphrase to user-facing text; hold out the original structured facts as gold truth.
- Include incomplete, contradictory, duplicate, long, urgent, code-switched English/Roman Urdu and prompt-injection examples.
- Keep evaluation seed separate from any examples used to design prompts or demos to reduce leakage.
- Score field-level exactness/precision/recall, type accuracy, missing-info recall, unsupported-field rate, latency and failure-to-manual behavior.
- Track rule/state tests separately from model evaluation; no LLM judges for basic deterministic invariants.
- If collecting pilot examples, obtain permission, minimize/redact private data, set retention rules, and do not send confidential records to free-tier AI providers without appropriate consent and contractual controls.

12. Security and operational requirements

- Password hashing (Argon2id or bcrypt) or OIDC; secure HttpOnly/SameSite cookies; CSRF defense if cookie auth; rate limit login and public ingress.
- Role/object-level authorization, separate approval from request originator, tenant-scoped SQL access; regression tests for horizontal IDOR.
- Secrets only as environment/IAM-managed values, never in source or browser; use scoped AWS IAM for S3/SES if added.
- Webhook signed over raw body with timestamp/replay window and stable event ID; store receipt transactionally before ACK.
- Prompt injection defenses: incoming text remains data; no arbitrary LLM tool permission; tool schemas and action constraints enforced server-side.
- Durable request state in PostgreSQL; queue delivery at-least-once with idempotent consumers and dead-letter/failed-job handling.
- Database backups plus one restore rehearsal; request IDs, structured logs, metrics, alerts for 5xx, queue lag, failed jobs, AI budget and disk space.
- HTTPS only; restrict EC2 security group; automated updates and image scanning; no public database/Redis ports.
- Document incident response, retention/deletion and provider data handling before onboarding real businesses.

13. Quality gates and definition of done

Gate	Acceptance criteria
Workflow behavior	Three end-to-end happy paths + reject/needs-info paths demonstrated without AI
Permissions	Tests reject self-approval, cross-organization access and unauthorized transitions
AI	Schema validation, unsupported-field refusal, fallback to manual review demonstrated; eval report reproducible
Reliability	Duplicate webhook and worker retry do not duplicate business effects; queue crash recovery tested
API	OpenAPI complete, documented error responses, pagination, idempotency and rate limits
Frontend	Responsive accessible key journeys incl. empty/loading/error/offline/AI unavailable states
Deployment	HTTPS demo URL, secrets absent from repo, CI pipeline, seeded demo, backup+restore tested
Documentation	README 10-minute local run, architecture diagram, ADRs, security tradeoffs, 3-minute demo recording
Real use	At least one external tester completes workflow and feedback recorded; real data optional, never claim deployed business adoption without evidence

14. Phased delivery roadmap

Phase	Output	Exit gate
0 — Foundation	Monorepo, Compose, CI, FastAPI+Next skeleton, DB migrations, seed 100	One command starts local environment, tests green
1 — Workflow core	Auth, roles, service/expense/leave state machines, API, inbox/detail/approval	All 3 workflows work manually end to end
2 — AI extraction	Groq/Gemini interfaces, typed extraction, clarification, eval runner	No hallucinated mandatory amount/vendor; fallback works
3 — Durable async	Redis worker, outbox, notifications, signed webhook, idempotency	Duplicate and crash-retry tests pass
4 — Insights	SQL dashboards, SLA checks, manager briefing	Metrics exact; briefing citeable and safe on provider outage
5 — Hardening	Permission/failure/security tests, logging, rate limits, load smoke, backup/restore	Acceptance gates pass
6 — Pilot	EC2 HTTPS, seed, docs, demo, feedback from 1–3 testers	External user completes a real workflow

Suggested pace: 6–9 focused weeks for a polished solo MVP, with significant variance depending on schedule, AI-assisted coding, and the amount of frontend polish. Avoid setting a public deadline before Phase 1 scope is tested.

15. Decisions and open questions

Decision / question	v1 position
Single tenant vs multitenant?	Single organization deployed, tenant-aware schema and authorization tests to avoid painful redesign
Authentication provider?	Local seeded credentials for development; production pilot OIDC or securely managed login before inviting outsiders
Queue?	Redis-backed worker + Postgres transactional outbox; make source-of-truth DB explicit
Workflow customization?	Seeded admin-editable thresholds; visual designer postponed
Attachments?	Text-only v1 until upload security and storage controls ready
Notification channel?	In-app first; email adapter next; no WhatsApp integration requirement
Provider free-tier limits?	Verify quotas/data terms; implement hard usage ceilings and graceful manual operation
Data localization/PII?	Synthetic only for demos until retention, consent, provider data processing reviewed
Name availability?	ActionDesk is a working name; trademark/domain/GitHub availability not yet verified

First coding milestone: exact checklist

7. Create monorepo folders and decision record for single-org pilot.
8. Create Docker Compose: Postgres, Redis, FastAPI, Next.js, worker; local .env.example only.
9. Define enum types/states, SQLAlchemy models and first Alembic migration.
10. Seed one organization, admin/manager/requester users and 10 minimal requests.
11. Implement login + request creation/list/detail with ownership and organization-scoped auth.
12. Implement transition engine as pure tested functions before wiring REST actions.
13. Add GitHub Actions test/lint/typecheck and a README with make dev / make test.
14. Only then introduce the AI adapter and large synthetic corpus.

Engineering mantra: Build the dependable request-and-approval product first. AI is an enhancement that may fail without breaking business operations.